BACHELORARBEIT im Studiengang

Wirtschaftsinformatik – Bachelor of Science (B.Sc.)

aus dem Fach/Modul: Wirtschaftsinformatik

gestellt von: Professor Dr. Peter Fettke

mit dem Thema: Prozessvorhersage mittels Transformer-Netzen am Beispiel der Fischertechnik APS

bearbeitet von

Name: Nicolas Christian Edelmann

Adresse: Reinhold-Zeller-Str. 19, 66386 St. Ingbert

Abgabetermin: .....

Spätester Beurteilungstermin: .....

Eidesstattliche Erklärung

Hiermit versichere an Eides statt, dass ich diese Arbeit selbständig angefertigt und keine anderen als die angegebenen Hilfsmittel verwendet habe. Alle wörtlichen oder sinngemäßen Entlehnungen sind deutlich als Solche gekennzeichnet.

Saarbrücken, June 09, 2026 _____

Nicolas Christian Edelmann

Contents

List of Figures

1

Introduction

Traditionally discovering knowledge about your business processes required combining estimates and manual data collection. This acquired knowledge would then be used to analyse, redesign and optimise said processes or to make process-external decisions (Di Francescomarino et al., 2026). This type of knowledge discovery is naturally error-prone and tends to be time-consuming.

Nowadays, more and more businesses rely on enterprise systems to store highly detailed process execution information, based on sensoric outputs, workflow data and machine logs. By applying various process mining techniques to these datasets, crucial knowledge can be extracted (Aalst, 2016), which can then be used in multiple business process lifecycles (Muehlen & Ho, 2006). During process *design*, analysis of historical data can provide comparison points, during process *monitoring* outliers and bottlenecks can be detected.

A subfield of process monitoring is predictive process monitoring. It is focused on predicting the behaviour of process executions under certain conditions. It can then provide information about the business goals, such as expected failure rates or execution time. If a process is structured into multiple events, *next event prediction* is concerned about the next events activity or surrounding metadata. Here, machine learning approaches have been on the rise, from using LSTMs (Camargo et al., 2019; Tax et al., 2017) to modified transformer architectures (Bukhsh et al., 2021; Ni et al., 2023).

In modern environments, process executions are inherently concurrent and parallel. As a result, given a running process execution, **the** next event is not clearly defined: Due to the parallel execution multiple events could be executed at the same time, or in an arbitrary order depending on system load. While a linear log collected by an enterprise system might exist, due to the parallel execution this order should not be fully enforced onto predictions. As an example, if a machine needs to produce both `A` and `B`, and then combine them, the imaginary prediction trace

```
Produce B -> Produce A -> Combine A and B
```

does not necessarily conflict with

```
Produce A -> Produce B -> Combine A and B
```

However there does exist a partial order of events within a process due to causal relationships between the events. Following the example from before, clearly

```
Combine A and B -> Produce B -> Produce A
```

is not feasable, as one can not combine non-existing products.

In this work, we present an approach to use transformer networks to solve the next event prediction problem, while working around the concurrency uncertainty by modelling it using the Heraklit infrastructure (Fettke & Reisig, 2020).

We evaluate this approach on the Fischertechnik **Agile Production Simulator** (APS)[1], providing the tools for end-to-end prediction. We extract process logs directly from the internal MQTT broker and then fit and train the model based on derived Heraklit steps. The model achieves an outstanding 92% accuracy on single next-event prediction. Similar results can still be achieved when simulating noisy MQTT logs by removing events or adding random events into the logs.

The structure of the thesis can be summarized as follows. Chapter 2 introduces the necessary background on Heraklit, Transformers and MQTT while providing information on theorical prediction approach.

Chapter 3 then explains the chosen modelling approach of the Fischertechnik APS case study using Heraklit. Continuing into the technical details, Chapter 4 provides a deeper dive into the details of the applied approach and the Fischertechnik data.

The case study model is lastly evaluated in Chapter 5 in various scenarios, before drawing conclusions, and discussing limitations and future work in Chapter 6.

[1] https://www.fischertechnik.de/de-de/industrie-und-hochschulen/technische-dokumente/simulieren/agile-production-simulation

2

Theory

In this chapter, we will present the theoretical and technical background of this work. This includes explaining the Heraklit modeling framework used to model the factory, the process prediction problem and the transformer architecture applied to solve it. Lastly, related work in the field of process prediction and process mining will be discussed.

2.1 Modelling with Heraklit

2.2 Process Prediction Problem

2.3 Transformers

What is this transformer?

2.3.1 How to apply this transformer theory to the process prediction problem?

2.4 MQTT

2.5 Related Work

3

Modelling

We want to evaluate our process prediction technique on the Fischer-Technik **Agile Production Simulator**[2]. Prior to the evaluation, we want to set the theoretical foundation of *what* the predictions mean. We do this by modelling the Heraklit (Fettke & Reisig, 2020) steps required to represent process models. These steps are later on predicted by our technique, thus it is crucial to have a clear understanding of what they represent and how they are defined.

Explain the Heraklit step modeling in background before

These steps can be grouped by the relevant factory modules:

- Automated Guided Vehicle (**AGV**): It autonomously transports workpieces between factory modules.
- High-Bay Warehouse (**HBW**): It serves as a storage system to the factory, designed with an automatic retrieval.
- Delivery And Pickup Station (**DPS**): It serves as the point of input and output of the factory. New workpieces are *inserted* here, processed workpieces are *shipped off*. The included NFC reader starts the tracking of workpieces across the factory.
- Drilling Station (**DRILL**): It performs a simulated drilling operation on the workpieces, picking them up from and dropping them onto the AGV.
- Milling Station (**MILL**): It performs a simulated milling operation on the workpieces, picking them up from and dropping them onto the AGV.
- Quality Control with AI (**AIQS**): Checks the quality of a workpiece using a camera and color sensor. If a failure occurred - represented by a erroneous print on the workpiece - the workpiece is discarded.
- Central Control Unit (**CCU**): It is the central controller of the factory. While it has no physical representation in the factory, it is the centralized decision maker of the factory, synchronizing the different distributed modules. Thus for our modelling purposes, it will be the **glue** that connects the modules together, while ensuring different processes for different piece types.

[2] https://www.fischertechnik.de/de-de/industrie-und-hochschulen/technische-dokumente/simulieren/agile-production-simulation

3.1 Heraklit Step Modelling

3.1.1 AGV

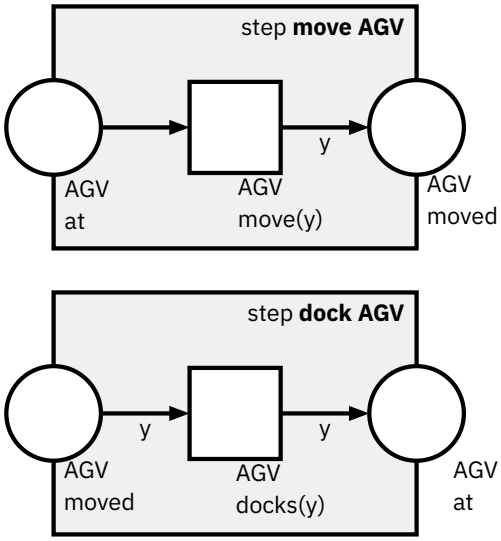


Figure 3.1: *AGV steps*

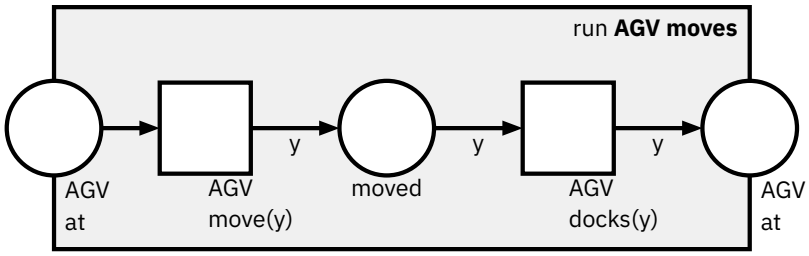


Figure 3.2: *AGV Move run:* move AGV ∘ dock AGV

These atomic steps are modeled with the variables and domains defined as

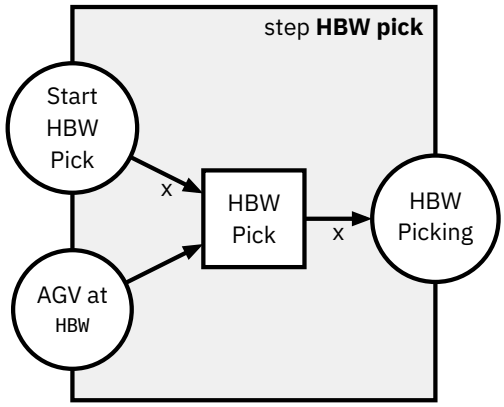
```
variable
y: process-module

domain
process-module: {DPS, HBW, DRILL, MILL, AIQS}
```

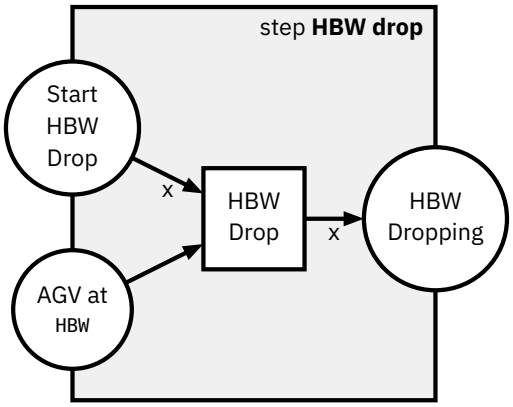
Important to notice is that the `AGV move` step transition is non-deterministic, as it does not consume the variable from the previous place, the input `AGV at`. It does however pass this decided variable onto the remaining parts of the process.

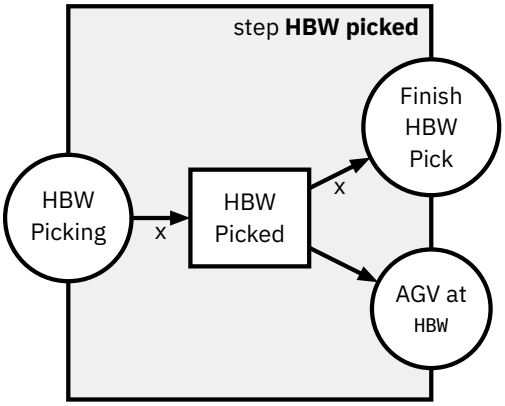
This will be relevant to ensure that the steps in the other modules can depend on the decision made in the `AGV move` step, as they depend on the AGVs position.

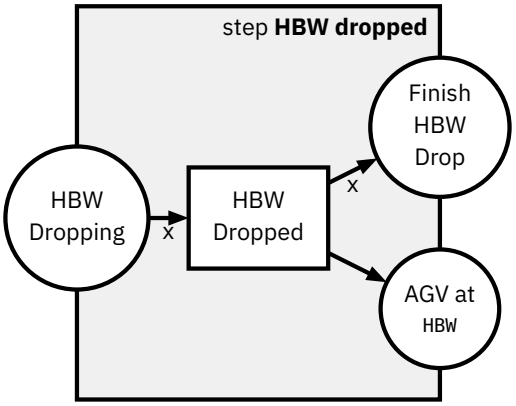
Figure 3.2 shows the composition of the two step atoms, showing the typical run that can be found in the factory logs.

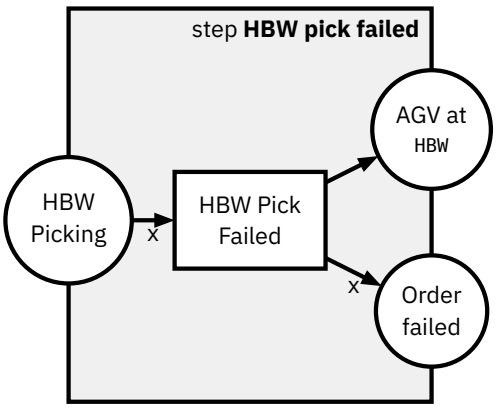
3.1.2 HBW











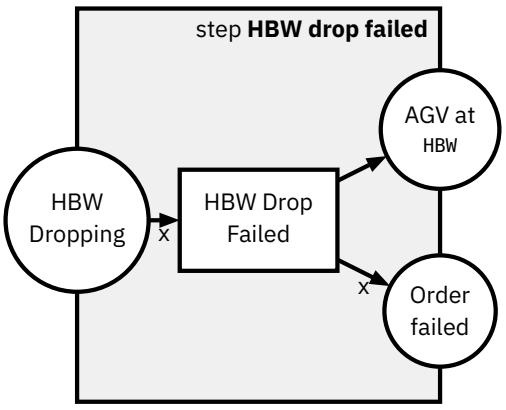


Figure 3.3: *HBW Pick steps*

Figure 3.4: *HBW Drop steps*

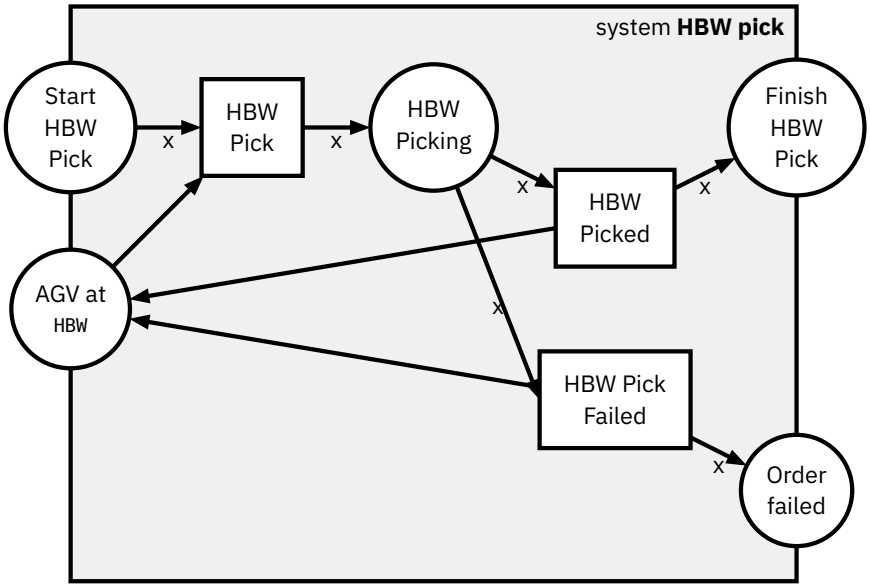


Figure 3.5: *HBW Pick model (Store into HBW)*

In the steps regarding the HBW actions in Figure 3.3 and Figure 3.4, we introduce the new variable x, defined as follows.

```
variable
x: workpiece

domain
workpiece: {blue, red, green}
```

The pattern of passing around the workpiece variable x will continue to appear in most other processing steps. This variable allows us to configure the processing steps to depend on the type of workpiece being processed, which is crucial to

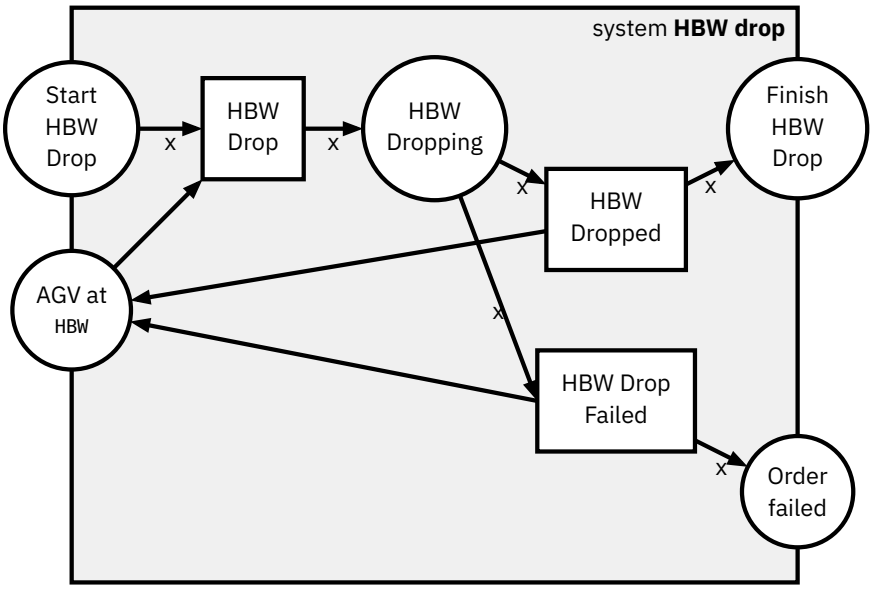


Figure 3.6: *HBW Drop model (Retrieve from HBW)*

ensure that the process prediction technique can learn to correctly predict the processing of different workpieces across the factory.

The HBW Pick system model (Figure 3.5) takes a workpiece from the AGV and stores it in the warehouse. The HBW Drop system model (Figure 3.6) performs the reverse operation, thus dropping a workpiece into the AGV.

Both system models are very similar, as they perform a similar operation, just in reverse. Surprisingly there is not much logic involved, as we decided to not model the explicit HBW state here, i.e. leaving out what is stored where.

3.1.3 DPS

The DPS needs to insert pieces into the factory and ship them out of the factory. The piece insertion into the factory is represented by a `DPS Drop`, dropping the piece onto the AGV from the loading bay. The shipping out of the factory by `DPS Pick`, picking the piece up from the AGV and dropping it off at the loading bay. Both steps are reading and writing to the NFC Tag on the piece, if present, providing a history of the piece across the factory. Similarly to the physical actuator control done by the Siemens SPS, we choose to not model the NFC tag explicitly, as the prediction will be on a higher level of abstraction.

Notably, the `DPS Drop` system (Figure 3.9) contains the only step that can introduce a new workpiece into the factory, thus it is the only step that can introduce a new variable assignment to `x`, as it is the only step that can introduce a new workpiece into the process. The remaining process steps will look similar to other upcoming Pick and Drop steps, however they will not be able to change or introduce the workpiece variable assignment.

Notice how the Figure 3.7 contains a failure case, where drop can fail. While most other failures will result in the order failing, at this point we don't have an active order with any part yet - the was just about the be arriving in our factory. Thus we only model it as a failed drop, the CCU will need to decide how to handle this failure, e.g. by retrying the drop or by cancelling operations.

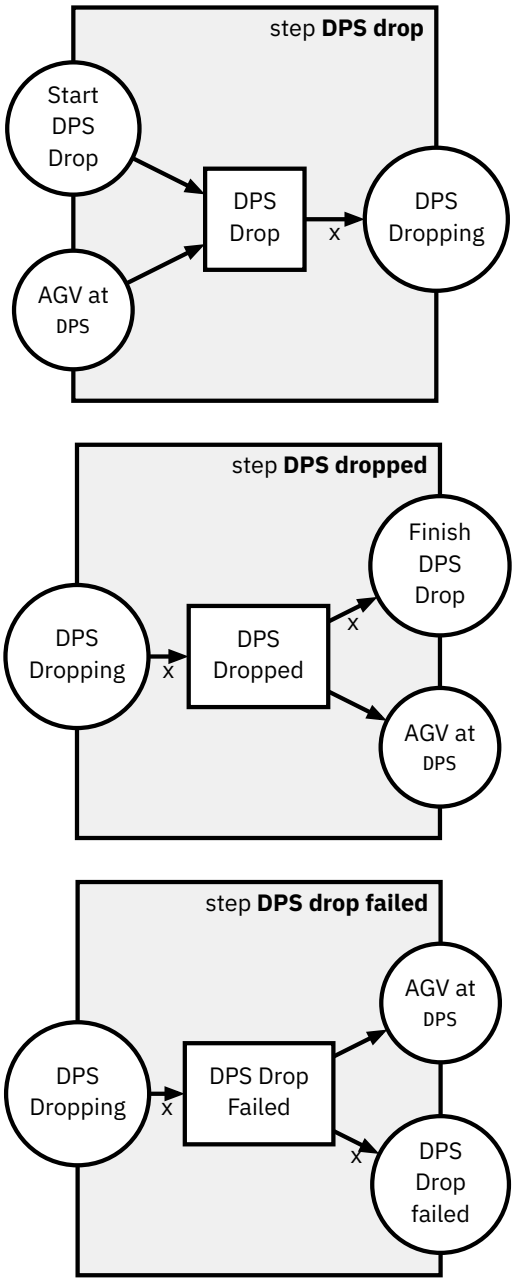


Figure 3.7: *DPS Drop steps*

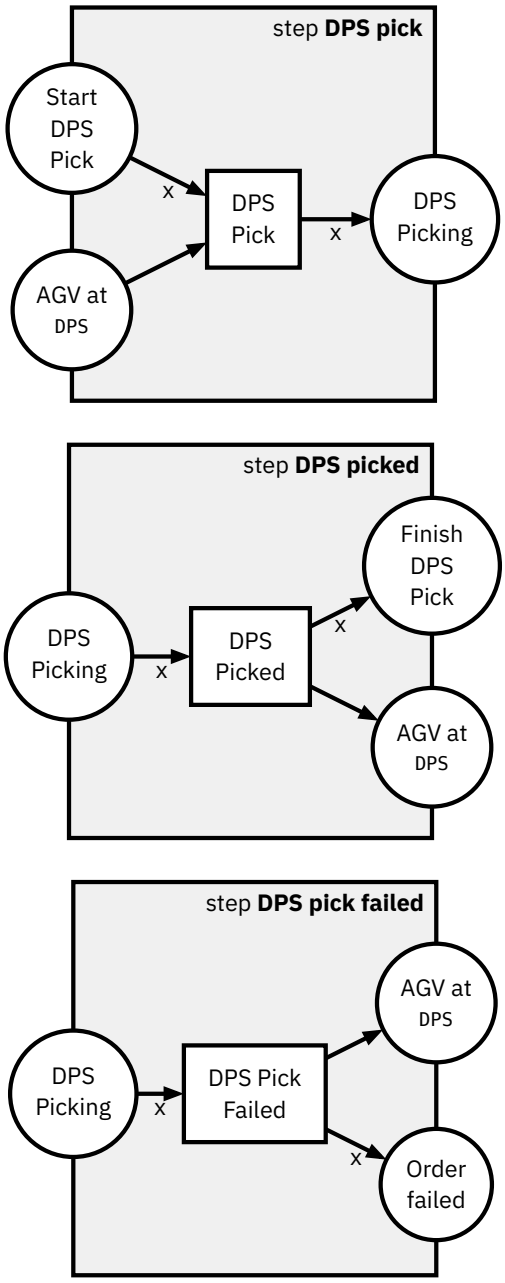


Figure 3.8: *DPS Pick steps*

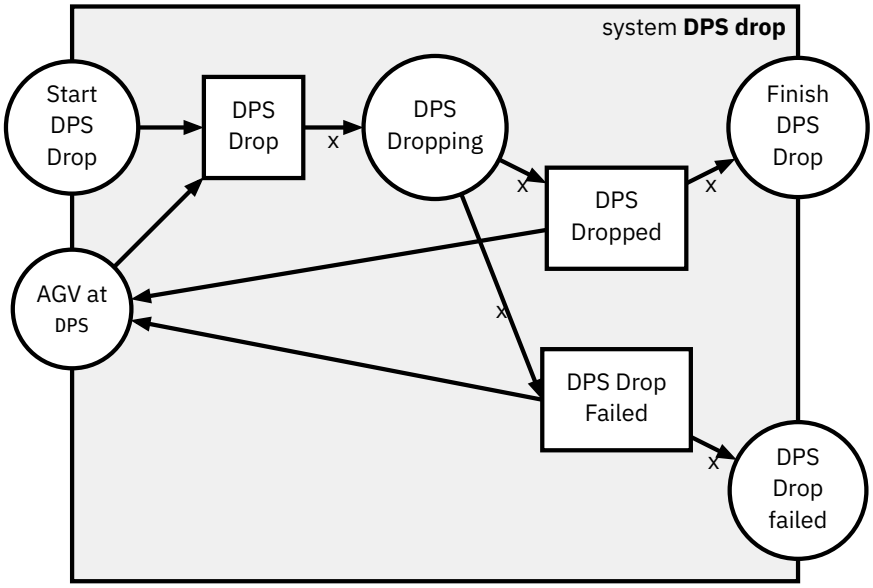


Figure 3.9: *DPS Drop model (Deliver into Factory)*

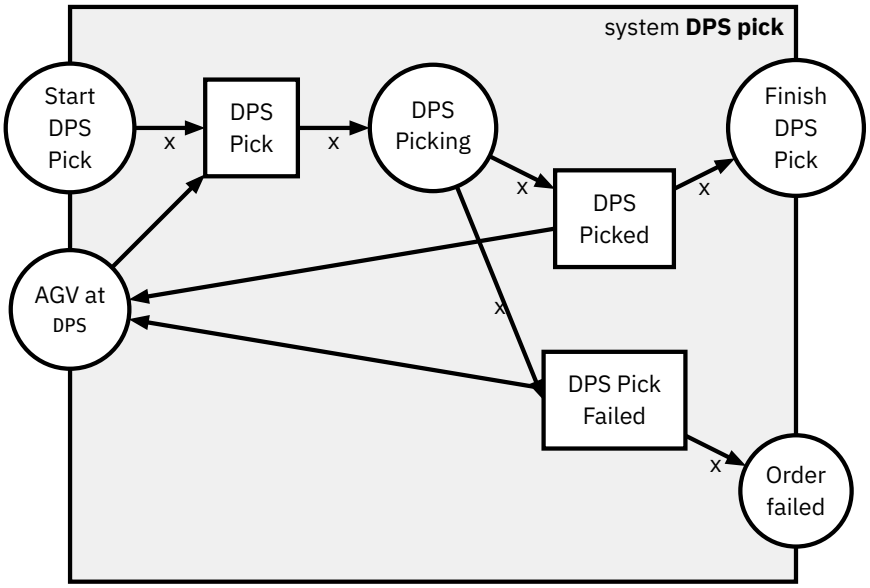


Figure 3.10: *DPS Pick model (Ship out of factory)*

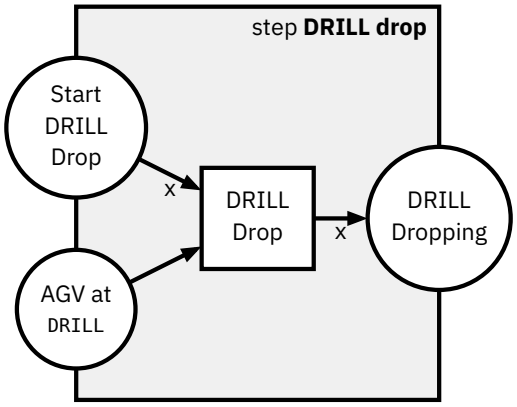
3.1.4 DRILL

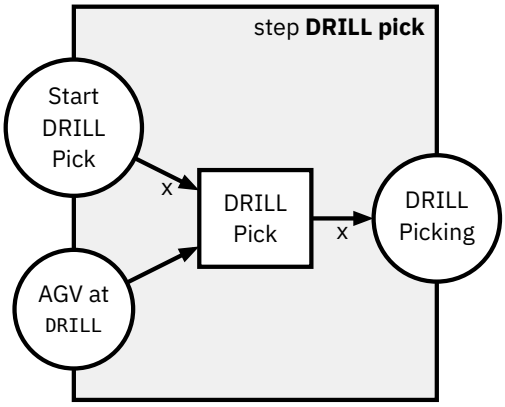
The drill module also has to interact with the AGV, thus we also introduce Pick (Figure 3.12) and Drop (Figure 3.11) steps for the drilling station. Additionally, the module can also perform a certain module-specific action, in this case the simulated drilling of a hole into the workpiece (Figure 3.13). The variables follow the same definitions as in previous module definitions.

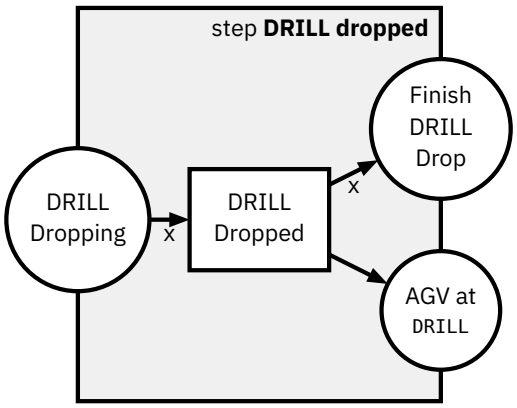
The Pick and Drop step definitions are defined completely analogous up to renaming to the ones of the HBW. We will see the same pattern in the coming modules as well, as the picking and droppping interactions with the AGV appear in all modules.

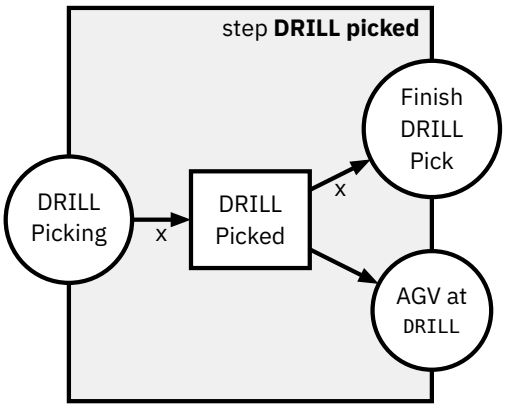
However, the drilling step is unique to the drill module, performing the actual processing of the worpiece at that station.

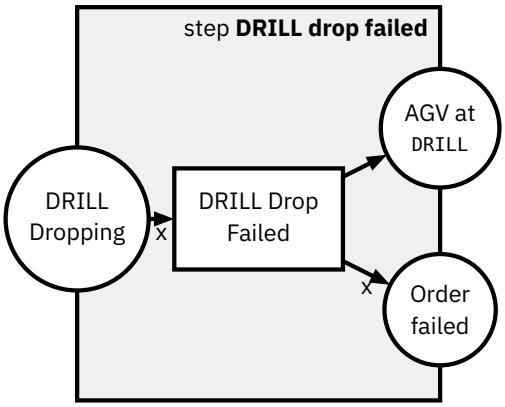
The typical compositions can be seen in Figure 3.14, Figure 3.15 and Figure 3.16. These system models combined provide a top-level view on the drilling station process.











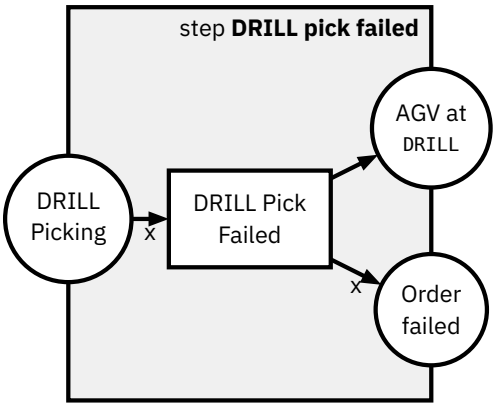


Figure 3.11: *DRILL Drop steps*

Figure 3.12: *DRILL Pick steps*

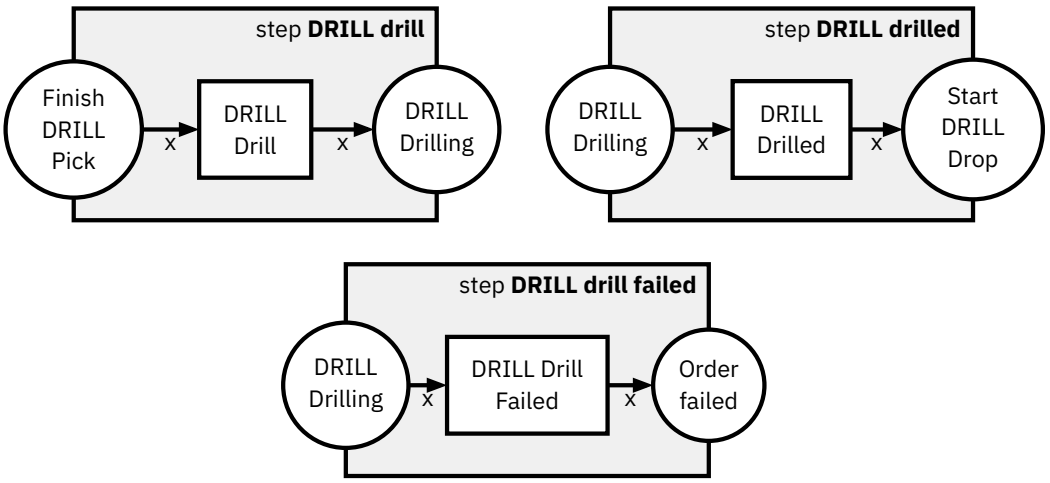


Figure 3.13: *DRILL Drill steps*

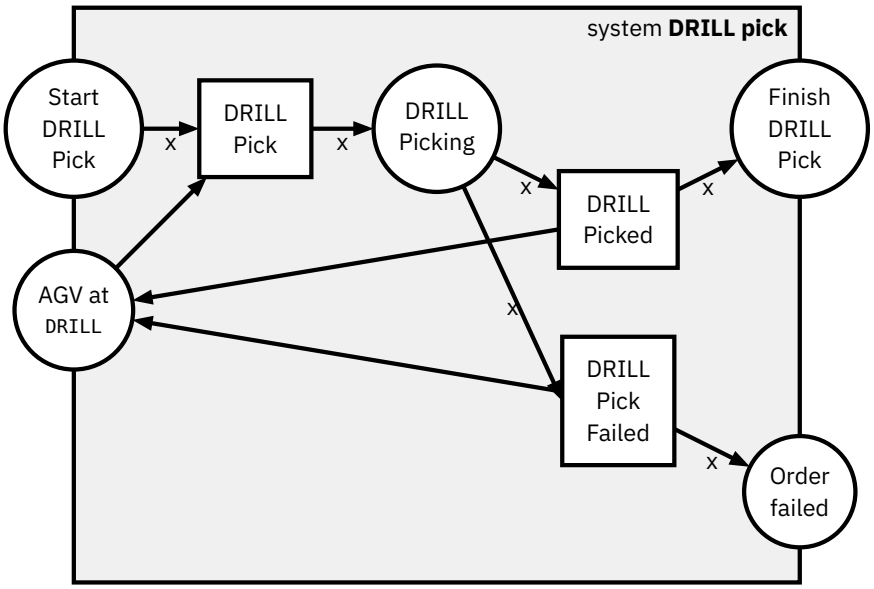


Figure 3.14: *DRILL Pick model*

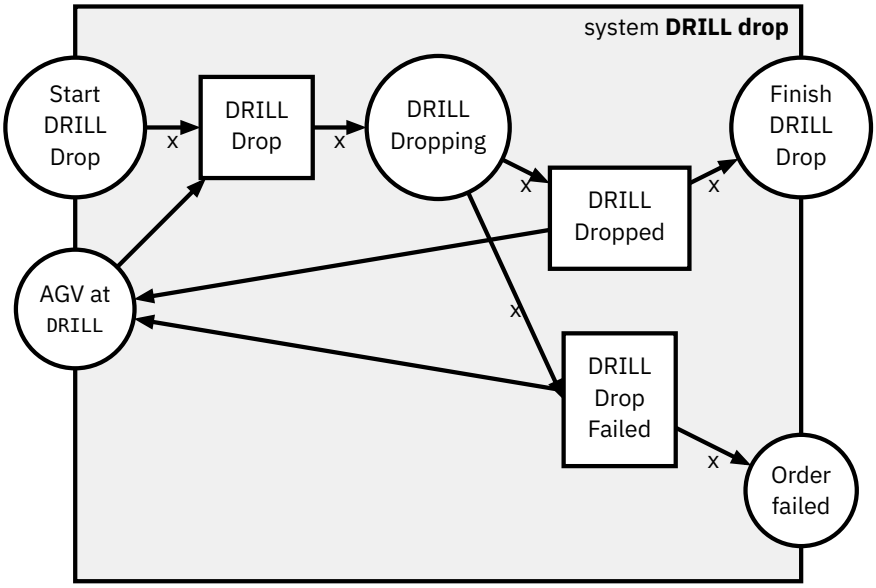


Figure 3.15: *DRILL Drop model*

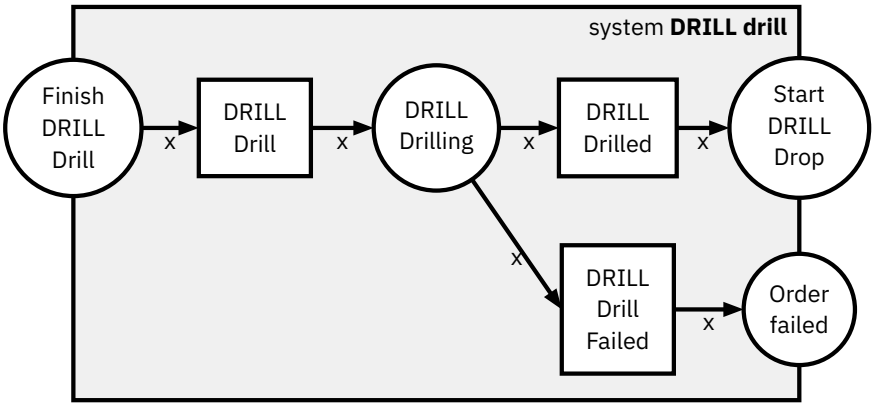
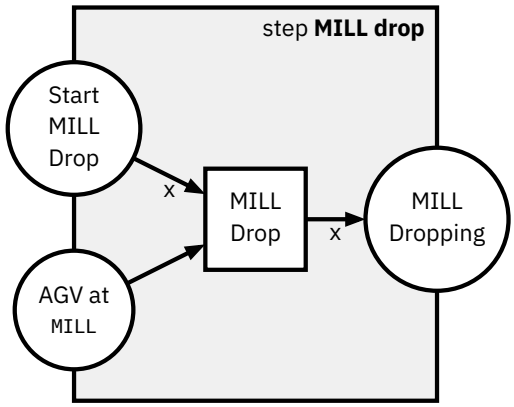


Figure 3.16: *DRILL Perform Drilling model*

3.1.5 MILL

The mill module performs the same function as the drill, only changing the simulated action to milling a groove into the workpiece instead of drilling.

Thus it contains the steps for picking (Figure 3.18), dropping (Figure 3.17) and milling (Figure 3.19). The system models are defined in Figure 3.20, Figure 3.21 and Figure 3.22.



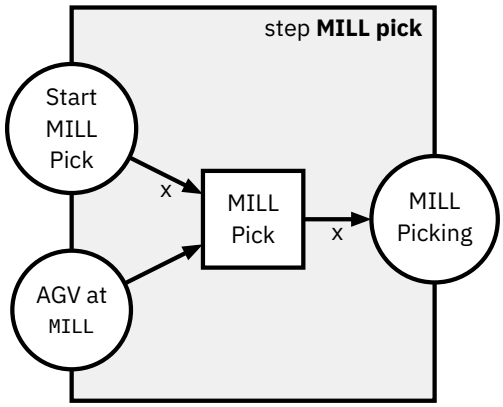
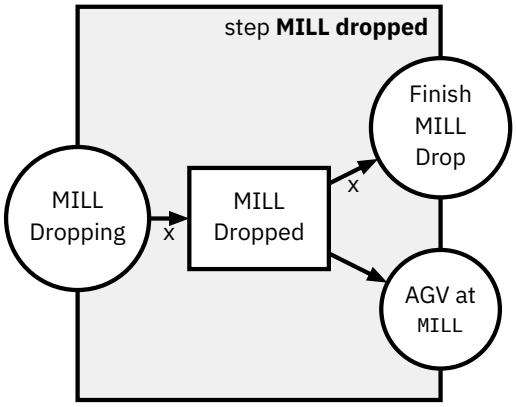
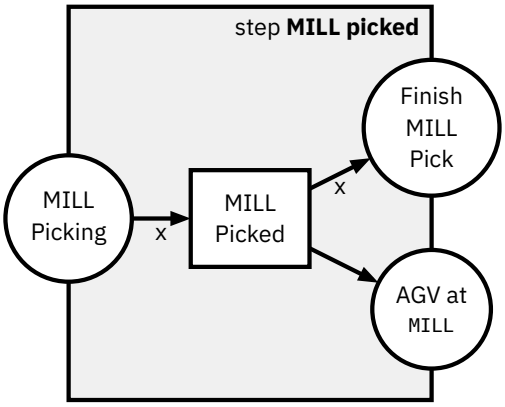
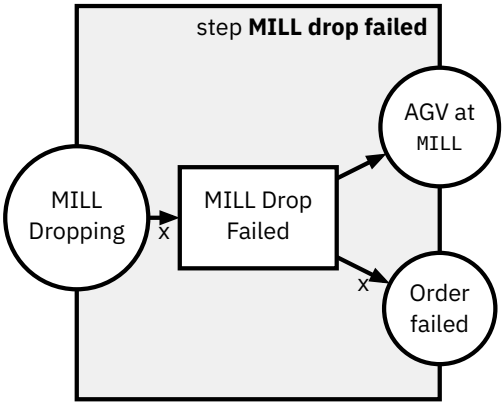


Figure 3.17: *MILL Drop steps*







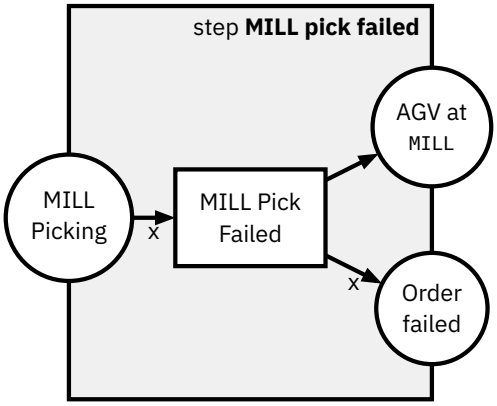


Figure 3.18: *MILL Pick steps*

3.1.6 AIQS

The last physical module is the AIQS, which provides a quality control checkpoint for all orders. Using a camera and a color sensor, it can detect faults (printed onto the pieces) and then discard faulty pieces into a trash chute, while passing good pieces on.

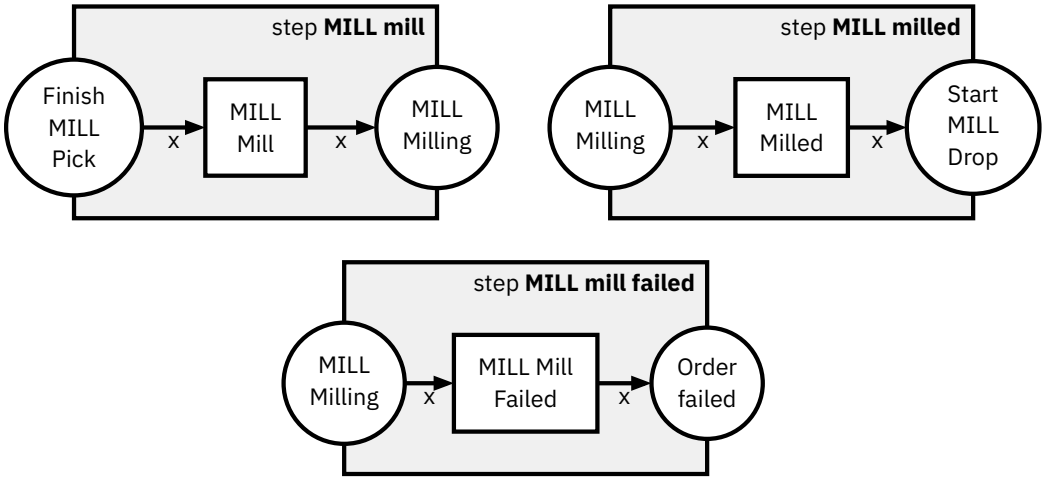


Figure 3.19: *MILL Mill steps*

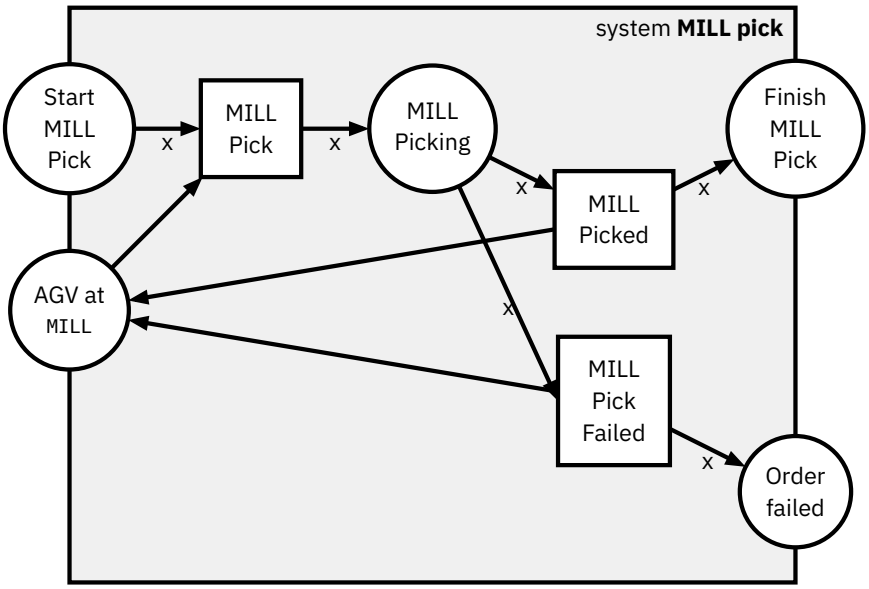


Figure 3.20: *MILL Pick model*

As with the previous two processing stations, it also contains Pick (Figure 3.24) and Drop (Figure 3.23) steps. Additionally, it implements its Check Quality (Figure 3.25) steps. The system models are defined in Figure 3.26, Figure 3.27 and Figure 3.28.

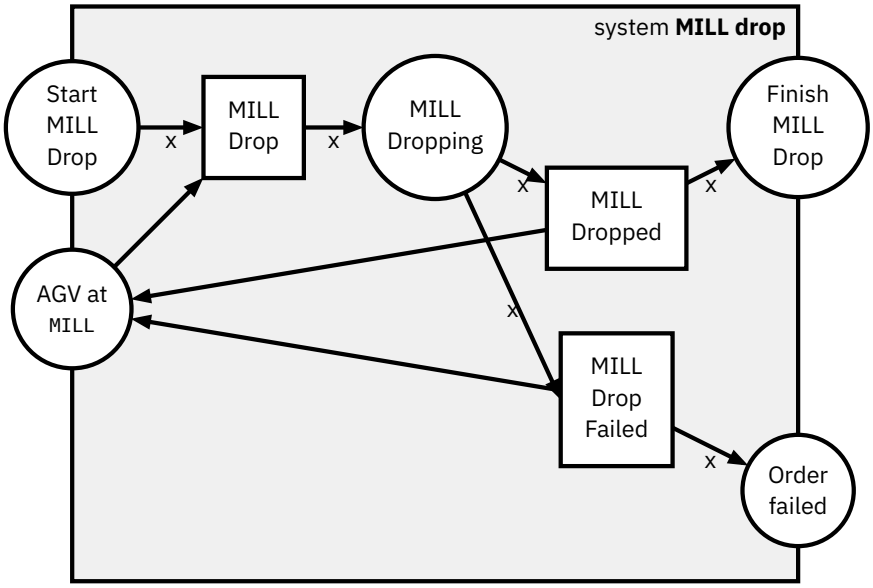


Figure 3.21: *MILL Drop model*

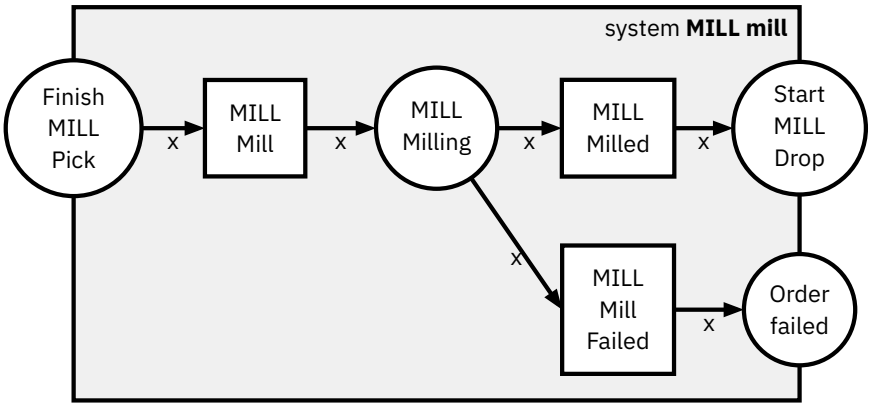
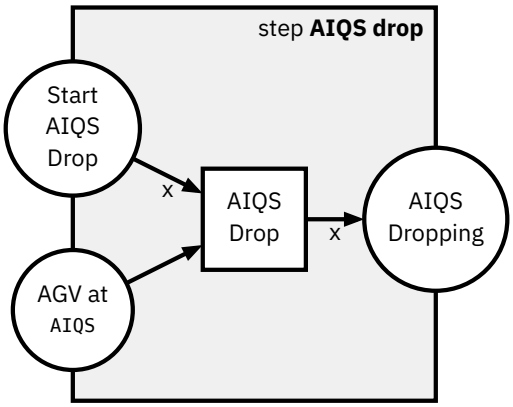
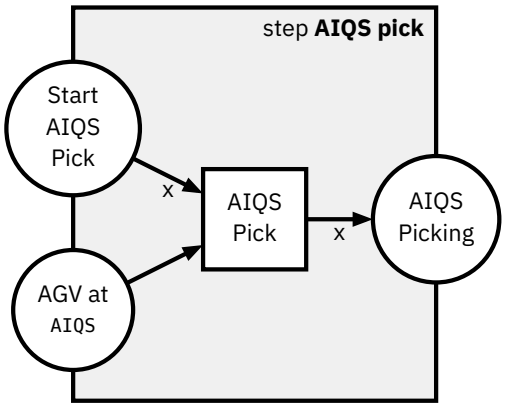
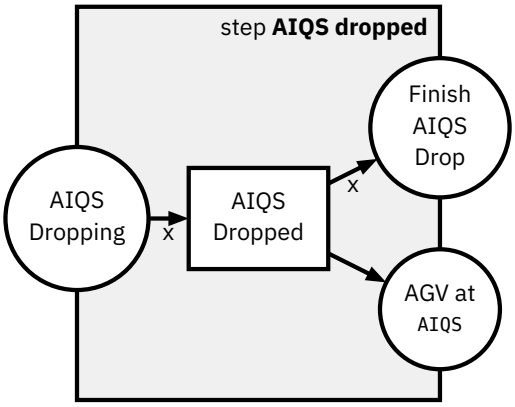
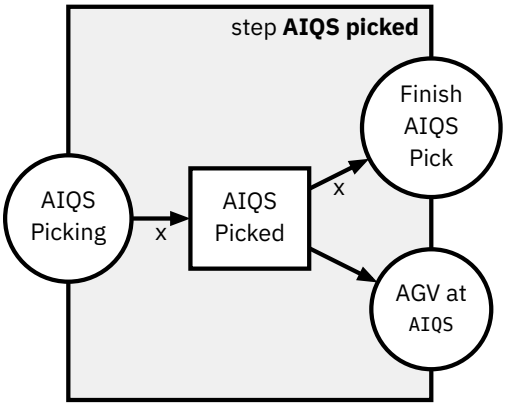


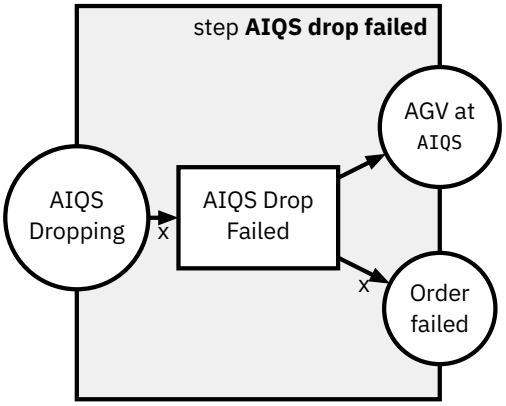
Figure 3.22: *MILL Perform Milling model*











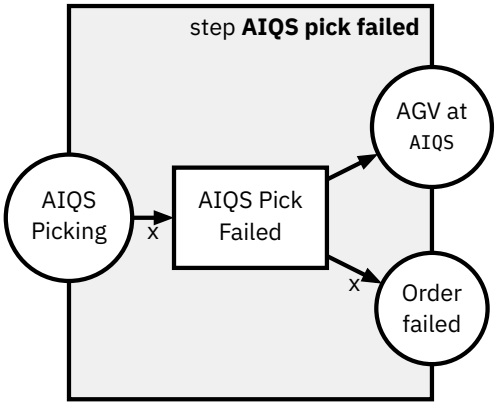


Figure 3.23: *AIQS Drop steps*

Figure 3.24: *AIQS Pick steps*

3.1.7 CCU

The CCU is the heart of the factory. It controls the interactions between the modules, taking the decisions on where the AGV should go and interacting with the factory owner.

Our goal is to model the steps extractable from the Fischertechnik MQTT Logs. These control steps, deciding what module action happens after the next, is

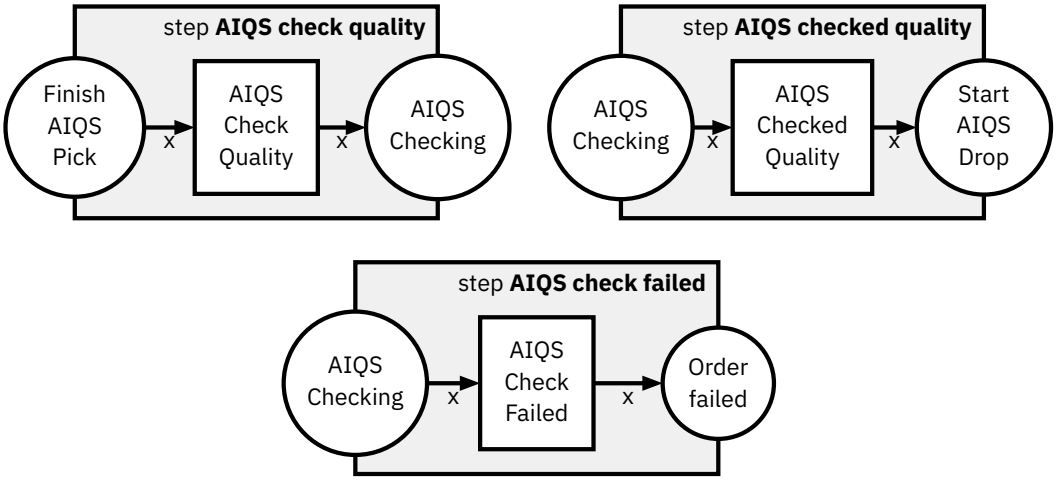


Figure 3.25: *AIQS Check Quality steps*

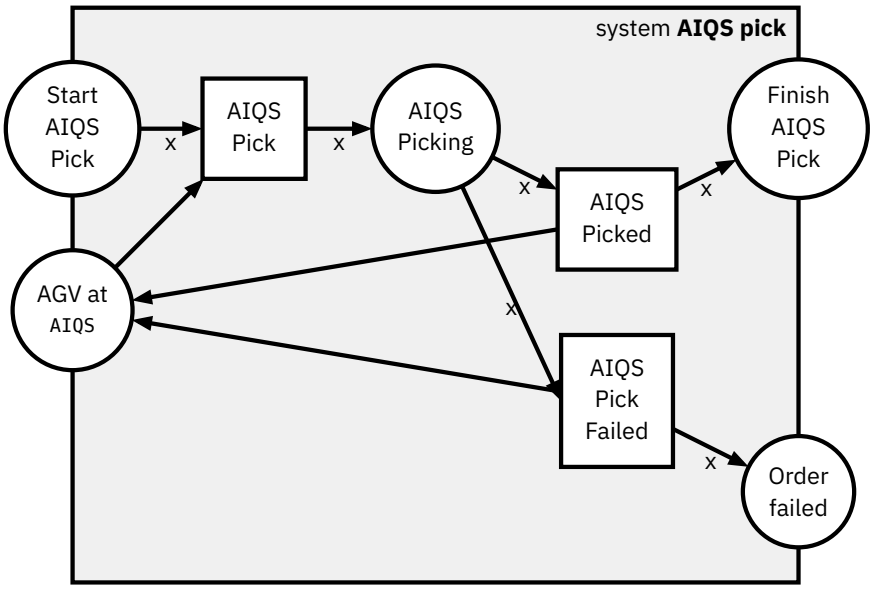


Figure 3.26: *AIQS Pick model*

implicit or **invisble** control. There is no control transition token to be found in the MQTT logs, the control can just be inferred from the interactions between the modules. This means that the CCU steps will not be present in the prediction tokens.

For a potential synthetic generation of runs modelling the different variations of runs, e.g. depending on the color of the workpiece, one could argue that these control steps must be meticulously designed. This would imply pre-modelling a specific order of module actions into the steps via the connecting places. An example can be seen in Figure 3.29.

This approach has multiple downsides. We trade the increased detail for **decreased flexibility**, as changes in the configuration for certain workpieces would require a new Heraklit step model. More importantly though, the tools to validate

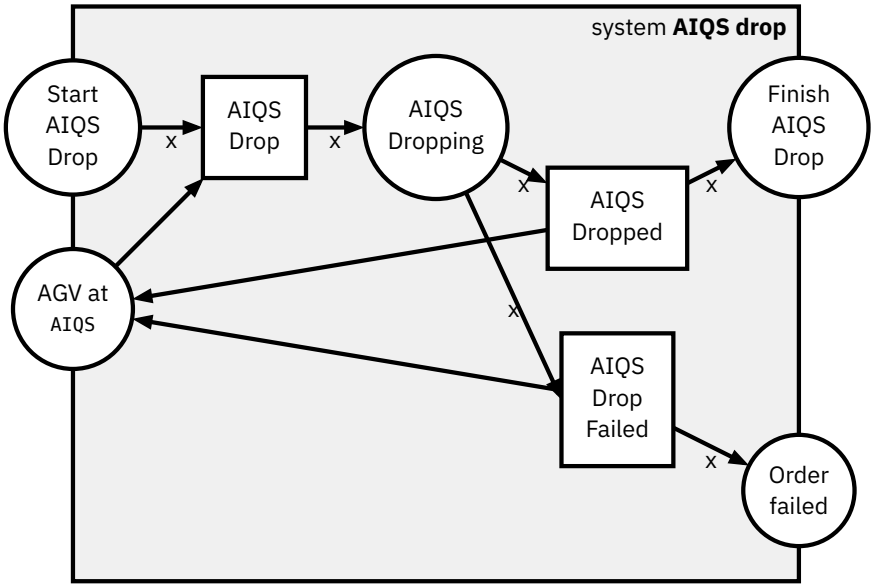


Figure 3.27: *AIQS Drop model*

model outputs would need to be capable of handling an **exponential number of steps** with increases in configurations and length of runs. As these control steps are not to be found within the logs, all matching control steps must be tried to be appended at any point of the validation, creating a huge search tree for validation.

We therefore decide not to model all the configurations explicitly. Instead, we only restrict our model to allow one module action to take place at a time. While this might seem counter-intuitive when looking at the distributed factory setting, here

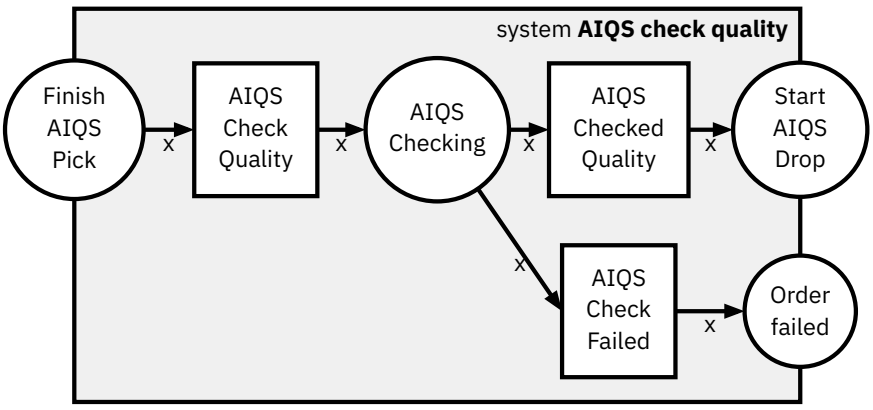


Figure 3.28: *AIQS Perform Quality Checking model*

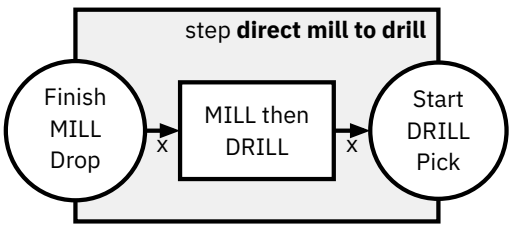


Figure 3.29: *Direct Module Interaction Modelling Approach*

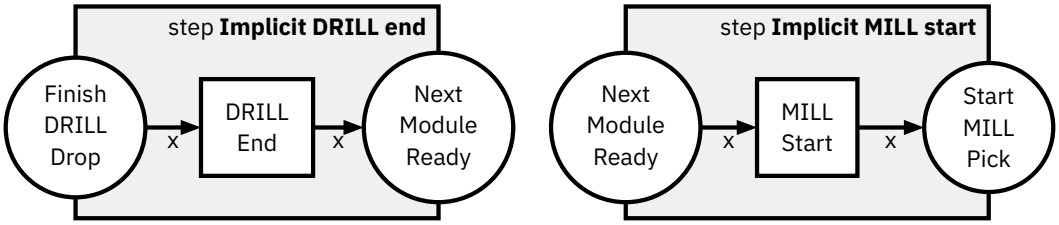


Figure 3.30: *Implicit Control of Mill to Drill steps*

we are only looking at the factory execution from the perspective of a singular workpiece. Since all parts of a singular workpiece are always only present in one module action, these actions don't need to be able to run concurrently.

Technically, this restriction is applied by creating a global place called `Next Module Ready`. Whenever a module wants to start, it needs to consume this place, whenever it is finished, it will fill the place again. Following the example from before, this creates the new steps in Figure 3.30.

This new design does not directly solve the issue of these control steps missing in the logs, but provides a simple solution. By composing DRILL Dropped ∘ Implicit DRILL end and Implicit MILL start ∘ MILL Pick it essentially just changes the interfaces of the module steps to have the `Next Module Ready` place instead of their respective `Start` and `Finish` places. These newly composed models can be then again composed directly via the `Next Module Ready`. As we know that before and after all module actions their steps will need their one matching `Implicit` step, we can pre-compose the control and module steps when talking about the actual logs.

For example, if the logs contain the steps

`DRILL Dropped` → `MILL Pick`

we can instead interpret this as

`DRILL Dropped` → `Implicit DRILL end` → `Implicit MILL start` → `MILL Pick`

as the implicit steps can be directly inferred.

At this point, one might wonder why we have not directly put the `Next Module Ready` step into the module steps. The reason for this decision is that we can keep the level of detail on the module basis, while essentially just having an interface wrapper to simplify implementation details later.

4

Implementation

This chapter provides the technical details of the implementation of our process prediction technique with encompassing information on the data collection, preprocessing and on the architecture and training of the final model.

4.1 Data Collection and Preprocessing

4.2 Model Architecture

4.3 Training Details

5

Evaluation

In this chapter we will first compare our prediction model with a random baseline, showing that our model is able to learn about the underlying process. We then evaluate different generalization capabilities of our model.

- Generalization

  TODO: Describe further evaluation...

- Caveats

6

Conclusion

TODO: Write conclusion. Repeat problem statement, then focus on results and implications

6.1 Limitations and Future Work

Bibliography

Aalst, W. van der. (2016). Data Science in Action. In *Process Mining: Data Science in Action* (pp. 3–23). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-662-49851-4_1

Bukhsh, Z. A., Saeed, A., & Dijkman, R. M. (2021). ProcessTransformer: Predictive Business Process Monitoring with Transformer Network. *Corr*. https://arxiv.org/abs/2104.00721

Camargo, M., Dumas, M., & González-Rojas, O. (2019). Learning accurate LSTM models of business processes. *International Conference on Business Process Management*, 286–302.

Di Francescomarino, C., Donadello, I., & Maggi, F. M. (2026). *Predictive Process Monitoring*. Springer.

Fettke, P., & Reisig, W. (2020). Modelling service-oriented systems and cloud services with Heraklit. *Corr*. https://arxiv.org/abs/2009.14040

Muehlen, M. z., & Ho, D. T.-Y. (2006). Risk Management in the BPM Lifecycle. In C. J. Bussler & A. Haller (Eds.), *Business Process Management Workshops: Business Process Management Workshops*.

Ni, W., Zhao, G., Liu, T., Zeng, Q., & Xu, X. (2023). Predictive Business Process Monitoring Approach Based on Hierarchical Transformer. *Electronics*, *12*(6). https://doi.org/10.3390/electronics12061273

Tax, N., Verenich, I., La Rosa, M., & Dumas, M. (2017). Predictive Business Process Monitoring with LSTM Neural Networks. In E. Dubois & K. Pohl (Eds.), *Advanced Information Systems Engineering: Advanced Information Systems Engineering*.